

Name : Sayan Das , Enrollment No: 12023052017057 , CSE(IOTCSBT) , Roll No: 57

Institute of Engineering & Management, Kolkata

Email ID: sayan.das2023@iem.edu.in

me.sayandas03@gmail.com

Video Demonstration Link : https://drive.google.com/drive/folders/1UMMiWthNj-LkAVPz_9NiJSs3IOQwF78x?usp=sharing

Module Name : Automation with Selenium

Experiment Title : Web Element Identification

Problem Statement :

To identify and locate web elements on a webpage using Selenium WebDriver and different locator strategies such as ID, Name, Class Name, Tag Name, Link Text, CSS Selector, and XPath, and to understand how these elements can be accessed and interacted with during web automation testing.

Objective :

To learn how to identify web elements using Selenium locators such as **ID, Name, Class Name, Tag Name, Link Text, CSS Selector, and XPath.**

Tools, Software, and Concepts Used

- **Python 3.x**
- **Selenium (latest version)**
- **Google Chrome / Mozilla Firefox**
- **WebDriver / Selenium WebDriver**
- **VS Code / PyCharm**
- Required Python packages:
 - **selenium**
 - **webdriver-manager**
- Concepts: **Web Elements, Locators, By class, CSS Selectors, XPath**

Source Code / Implementation Steps

1. Install the required packages using pip.
2. Import Selenium and WebDriver-related modules.
3. Open the required webpage using Chrome/Firefox.
4. Identify web elements using different Selenium locators.
5. Perform actions such as click, enter text, and select elements.
6. Verify the identified elements and close the browser using driver.quit().

Source Code:

```
from selenium import webdriver

from selenium.webdriver.common.by import By

import time

driver = webdriver.Chrome()

driver.maximize_window()

driver.get("https://testautomationpractice.blogspot.com/")

time.sleep(2)

print("=" * 70)

print("INPUT ELEMENTS")

print("=" * 70)

inputs = driver.find_elements(By.TAG_NAME, "input")

for i, el in enumerate(inputs, start=1):

    print(f'{i}. id='{el.get_attribute('id')}' | name='{el.get_attribute('name')}' "

          f"| type='{el.get_attribute('type')}' | placeholder='{el.get_attribute('placeholder')}'")

print("\n" + "=" * 70)

print("TEXTAREA ELEMENTS")

print("=" * 70)

textareas = driver.find_elements(By.TAG_NAME, "textarea")

for i, el in enumerate(textareas, start=1):

    print(f'{i}. id='{el.get_attribute('id')}' | name='{el.get_attribute('name')}' "

          f"| class='{el.get_attribute('class')}'")

print("\n" + "=" * 70)

print("SELECT (dropdown) ELEMENTS")

print("=" * 70)

selects = driver.find_elements(By.TAG_NAME, "select")
```

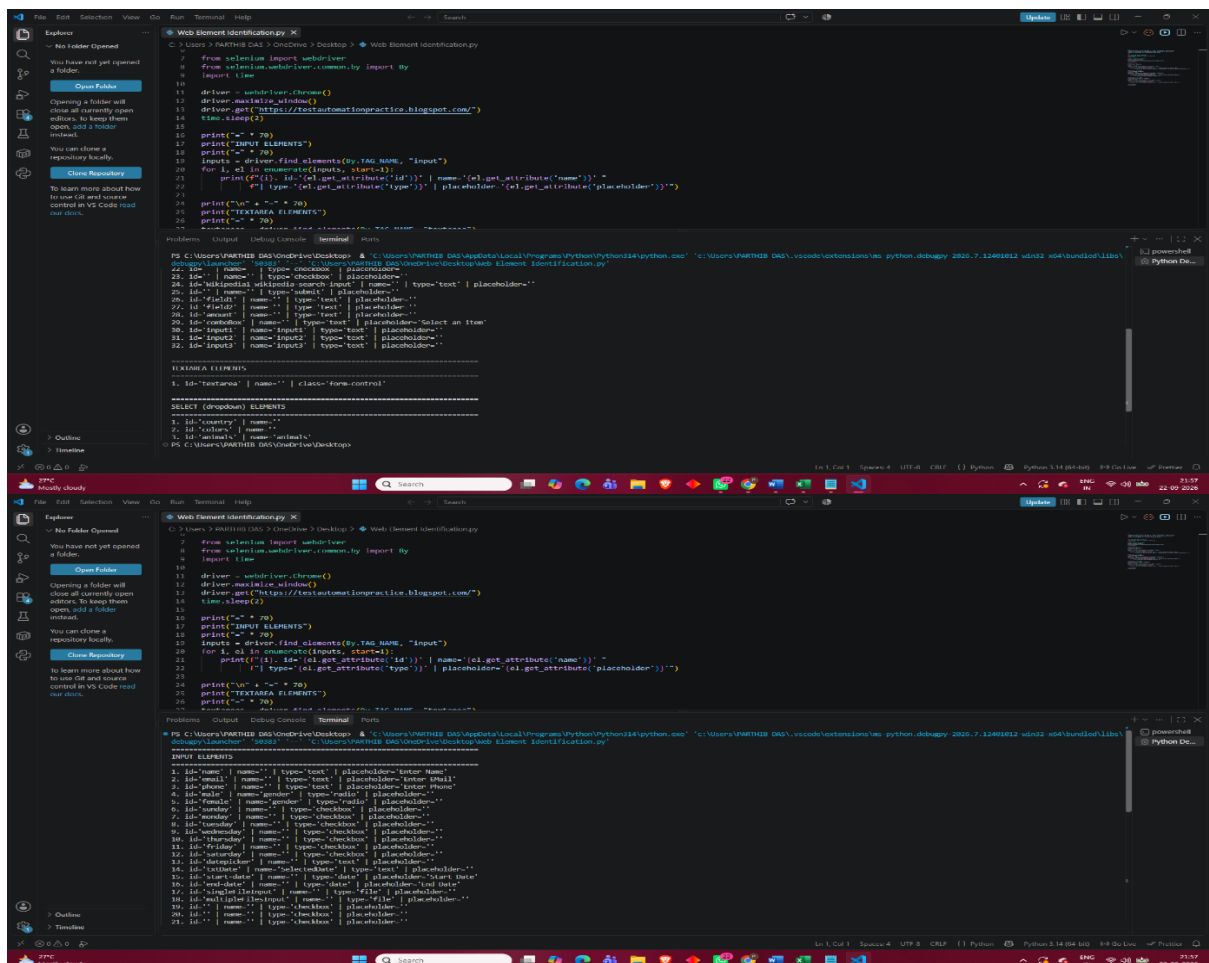
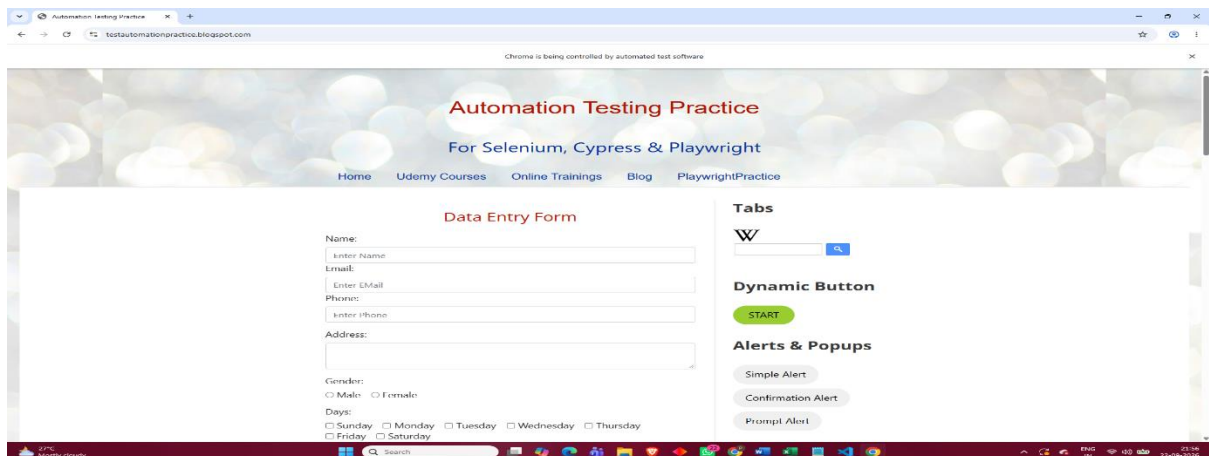
for i, el in enumerate(selects, start=1):

```
print(f'{i}. id='{el.get_attribute('id')} | name='{el.get_attribute('name')}')"
```

time.sleep(3)

driver.quit()

Output with clearly labelled screenshots:



Result: Web elements were successfully identified using different Selenium locator strategies.

Observation: Selenium provides multiple locators such as ID, Name, Class Name, Tag Name, Link Text, CSS Selector, and XPath for locating elements.

Conclusion: The experiment helped understand the identification and basic interaction with web elements using Selenium WebDriver.

Experiment Title : Multiple Element Identification

Problem Statement:

To identify and locate multiple web elements on a webpage using Selenium WebDriver and suitable locator strategies, and to retrieve and interact with a collection of similar elements efficiently during web automation testing.

Objective

To learn how to identify multiple web elements using Selenium's `find_elements()` method with locators such as **ID, Name, Class Name, Tag Name, CSS Selector, and XPath**.

Tools, Software, and Concepts Used

- **Python 3.x**
- **Selenium (latest version)**
- **Google Chrome / Mozilla Firefox**
- **Selenium WebDriver**
- **VS Code / PyCharm**
- Required Python packages:
 - `selenium`
 - `webdriver-manager`
- Concepts: **Multiple Web Elements, `find_elements()`, Locators, CSS Selectors, XPath**

Source Code / Implementation Steps

1. Install the required Python packages using `pip`.
2. Import Selenium and the required WebDriver modules.
3. Open the required webpage using Selenium WebDriver.
4. Use the `find_elements()` method to locate multiple elements.
5. Store the identified elements in a list.
6. Iterate through the list to access or perform actions on each element.
7. Verify the identified elements and close the browser using `driver.quit()`.

Source Code:

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```

from selenium.webdriver.support.ui import WebDriverWait

from selenium.webdriver.support import expected_conditions as EC

import time

driver = webdriver.Chrome()

driver.maximize_window()

driver.get("https://testautomationpractice.blogspot.com/")

wait = WebDriverWait(driver, 10)

# 1) Find all <a> (link) elements on the page and print their text + href

links = driver.find_elements(By.TAG_NAME, "a")

print(f"Total links found: {len(links)}\n")

for index, link in enumerate(links, start=1):

    text = link.text.strip()

    href = link.get_attribute("href")

    if text: # skip empty/icon-only links

        print(f"{index}. Text: '{text}' -> URL: {href}")

# 2) Find all <input> elements and print their 'type' attribute

print("\n--- Input field types on the page ---")

inputs = driver.find_elements(By.TAG_NAME, "input")

print(f"Total input elements found: {len(inputs)}")

for i, inp in enumerate(inputs, start=1):

    print(f"{i}. type = {inp.get_attribute('type')}, id = {inp.get_attribute('id')}")

# 3) Find all checkboxes (elements of the same "type") and tick each one

checkboxes = driver.find_elements(By.CSS_SELECTOR, "input[type='checkbox']")

print(f"\nTotal checkboxes found: {len(checkboxes)}")

for cb in checkboxes:

    if cb.is_displayed() and not cb.is_selected():

        driver.execute_script("arguments[0].scrollIntoView(true);", cb)

```

```

        cb.click()

print("All visible checkboxes have been checked.")

time.sleep(2)

driver.quit()

```

Output with clearly labelled screenshots:

```

1. Text: 'Home' -> URL: http://testautomationpractice.blogspot.com/
2. Text: 'Udemy Courses' -> URL: https://www.pavanonlinetrainings.com/#udemy
3. Text: 'Online Trainings' -> URL: https://www.pavanonlinetrainings.com/
4. Text: 'Blog' -> URL: https://www.pavantestingtools.com/
5. Text: 'PlaywrightPractice' -> URL: https://testautomationpractice.blogspot.com/p/playwrightpractice.html
7. Text: 'Data Entry Form' -> URL: https://testautomationpractice.blogspot.com/2018/09/automation-form.html
8. Text: 'Home' -> URL: https://testautomationpractice.blogspot.com/
9. Text: 'Posts (Atom)' -> URL: https://testautomationpractice.blogspot.com/feeds/posts/default
10. Text: '1' -> URL: https://testautomationpractice.blogspot.com/#
11. Text: '2' -> URL: https://testautomationpractice.blogspot.com/#
12. Text: '3' -> URL: https://testautomationpractice.blogspot.com/#
13. Text: '4' -> URL: https://testautomationpractice.blogspot.com/#
17. Text: 'Apple' -> URL: https://www.apple.com/
18. Text: 'Lenovo' -> URL: https://www.lenovo.com/
19. Text: 'Dell' -> URL: https://www.dell.com/
20. Text: 'Errorcode 400' -> URL: http://www.deadlinkcity.com/error-page.asp?e=400
21. Text: 'Errorcode 401' -> URL: http://www.deadlinkcity.com/error-page.asp?e=401
22. Text: 'Errorcode 403' -> URL: http://www.deadlinkcity.com/error-page.asp?e=403
23. Text: 'Errorcode 404' -> URL: http://www.deadlinkcity.com/error-page.asp?e=404
24. Text: 'Errorcode 408' -> URL: http://www.deadlinkcity.com/error-page.asp?e=408
25. Text: 'Errorcode 500' -> URL: http://www.deadlinkcity.com/error-page.asp?e=500
26. Text: 'Errorcode 502' -> URL: http://www.deadlinkcity.com/error-page.asp?e=502
27. Text: 'Errorcode 503' -> URL: http://www.deadlinkcity.com/error-page.asp?e=503
28. Text: 'Home' -> URL: http://testautomationpractice.blogspot.com/
29. Text: 'Hidden Elements & AJAX' -> URL: https://testautomationpractice.blogspot.com/p/gui-elements-ajax-hidden.html
30. Text: 'Download Files' -> URL: https://testautomationpractice.blogspot.com/p/download-files_25.html
31. Text: 'Youtube' -> URL: https://www.youtube.com/@sdetpavan/videos
32. Text: 'merrymoonmary' -> URL: http://www.istockphoto.com/portfolio/merrymoonmary?platform=blogger
33. Text: 'Blogger' -> URL: https://www.blogger.com/

--- Input field types on the page ---
Total input elements found: 32
1. type = text, id = name
2. type = text, id = email
3. type = text, id = phone
4. type = radio, id = male
5. type = radio, id = female
6. type = checkbox, id = sunday
7. type = checkbox, id = monday
8. type = checkbox, id = tuesday
9. type = checkbox, id = wednesday
10. type = checkbox, id = thursday
11. type = checkbox, id = friday
12. type = checkbox, id = saturday
13. type = text, id = datePicker
14. type = text, id = txtDate
15. type = date, id = start-date
16. type = date, id = end-date
17. type = file, id = singlefileInput

```

```

--- Input field types on the page ---
Total input elements found: 32
1. type = text, id = name
2. type = text, id = email
3. type = text, id = phone
4. type = radio, id = male
5. type = radio, id = female
6. type = checkbox, id = sunday
7. type = checkbox, id = monday
8. type = checkbox, id = tuesday
9. type = checkbox, id = wednesday
10. type = checkbox, id = thursday
11. type = checkbox, id = friday
12. type = checkbox, id = saturday
13. type = text, id = datePicker
14. type = text, id = txtDate
15. type = date, id = start-date
16. type = date, id = end-date
17. type = file, id = singlefileInput
18. type = file, id = multipleFilesInput
19. type = checkbox, id =
20. type = checkbox, id =
21. type = checkbox, id =
22. type = checkbox, id =
23. type = checkbox, id =
24. type = text, id = Wikipedia1_wikipedia-search-input
25. type = submit, id =
26. type = text, id = field1
27. type = text, id = field2
28. type = text, id = amount
29. type = text, id = comboBox
30. type = text, id = input1
31. type = text, id = input2
32. type = text, id = input3

Total checkboxes found: 12
All visible checkboxes have been checked.
PS C:\Users\PARTHIB DAS\OneDrive\Desktop>

```

Result, Observation, and Conclusion

Result: Multiple web elements were successfully identified and accessed using Selenium WebDriver.

Observation: The `find_elements()` method returns a list of all matching web elements, allowing multiple similar elements to be processed together.

Conclusion: The experiment helped understand how to locate and work with multiple web elements efficiently using Selenium.

Experiment Title : Child Nodes Using CSS

Problem Statement

To identify and locate child elements within a parent web element using **CSS Selectors** in Selenium WebDriver and understand how CSS child selectors can be used for precise web element identification.

Objective

To learn how to locate **child nodes/elements** using CSS selectors such as `parent > child` and perform basic interactions with them using Selenium.

Tools, Software, and Concepts Used

- **Python 3.x**
- **Selenium (latest version)**
- **Google Chrome / Mozilla Firefox**
- **Selenium WebDriver**
- **VS Code / PyCharm**
- Required Python packages:
 - `selenium`
 - `webdriver-manager`
- Concepts: **Child Nodes, Parent-Child Relationship, CSS Selectors, `find_element()`, `find_elements()`**

Source Code / Implementation Steps

1. Install the required Python packages using `pip`.
2. Import Selenium and the required WebDriver modules.
3. Open the required webpage using Selenium WebDriver.
4. Identify the parent element using a suitable CSS selector.
5. Use the `>` CSS child combinator to locate its direct child element.
6. Perform an action such as clicking or retrieving the child element.
7. Verify the result and close the browser using `driver.quit()`.

Source Code:

```
from selenium import webdriver
```

```

from selenium.webdriver.common.by import By

import time

driver = webdriver.Chrome()

driver.maximize_window()

driver.get("https://testautomationpractice.blogspot.com/")

time.sleep(2)

def describe(label, by, value):

    try:

        el = driver.find_element(by, value)

        print(f"{label}: FOUND")

        print(f"  tag='{el.tag_name}' id='{el.get_attribute('id')}' "
              f"class='{el.get_attribute('class')}'")

        # also show the parent, useful for building child selectors

        parent = el.find_element(By.XPATH, "..")

        print(f"  PARENT -> tag='{parent.tag_name}' id='{parent.get_attribute('id')}' "
              f"class='{parent.get_attribute('class')}'")

    except Exception as e:

        print(f"{label}: NOT FOUND ({e.__class__.__name__})")

    print("-" * 70)

# "Point Me" hover button

describe("Point Me button", By.XPATH, "//*[contains(text(),'Point Me')]")

# "Mobiles" / "Laptops" dropdown links

describe("Mobiles link", By.LINK_TEXT, "Mobiles")

describe("Laptops link", By.LINK_TEXT, "Laptops")

# "Copy Text" button

describe("Copy Text button", By.XPATH, "//*[contains(text(),'Copy Text')]")

# Static web table

```

```
describe("Static table", By.XPATH,

    "//table[.//td[contains(text(),'Learn Selenium')]]")

time.sleep(3)

driver.quit()
```

Output with clearly labelled screenshots:

```
Point Me button: FOUND
  tag='button' id='' class='dropbtn'
  PARENT -> tag='div' id='' class='dropdown'
-----
Mobiles link: NOT FOUND (NoSuchElementException)
-----
Laptops link: NOT FOUND (NoSuchElementException)
-----
Copy Text button: FOUND
  tag='button' id='' class=''
  PARENT -> tag='div' id='' class='widget-content'
-----
Static table: FOUND
  tag='table' id='' class=''
  PARENT -> tag='div' id='' class='widget-content'
-----
PS C:\Users\PARTHIB DAS\OneDrive\Desktop>
```

Result: Child elements were successfully identified using CSS child selectors.

Observation: The > CSS selector helps locate **direct child elements** of a specified parent element.

Conclusion: The experiment helped understand how CSS child selectors can be used for accurate and efficient web element identification in Selenium.

Experiment Title : Child Nodes Using CSS

Problem Statement

To identify and locate child nodes of a parent web element using CSS selectors in Selenium WebDriver for accurate web element identification and interaction.

Objective

To understand and implement CSS child selectors for locating direct child elements of a parent element using Selenium.

Tools, Software, and Concepts Used

- Python 3.x

- **Selenium (latest version)**
- **Google Chrome / Mozilla Firefox**
- **Selenium WebDriver**
- **VS Code / PyCharm**
- Python packages:
 - selenium
 - webdriver-manager
- Concepts: **CSS Selectors, Child Nodes, Parent-Child Relationship, `find_element()`, `find_elements()`**

Source Code / Implementation Steps

1. Install Selenium and WebDriver Manager.
2. Import the required Selenium modules.
3. Launch the browser using Selenium WebDriver.
4. Open the required webpage.
5. Identify the parent element using a CSS selector.
6. Use the > child selector to locate its direct child node.
7. Perform the required action on the identified child element.
8. Verify the output and close the browser.

Source Code :

```
from selenium import webdriver

from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import WebDriverWait

from selenium.webdriver.support import expected_conditions as EC

import time

driver = webdriver.Chrome()

driver.maximize_window()

driver.get("https://testautomationpractice.blogspot.com/")

wait = WebDriverWait(driver, 10)

name_field = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "input#name")))

name_field.send_keys("Selenium Tester")

print("CSS id selector -> Name field filled")

date_fields = driver.find_elements(By.CSS_SELECTOR, "input[id^='date']")

print(f"Elements whose id starts with 'date': {len(date_fields)}")

for d in date_fields:

    print(" ->", d.get_attribute("id"))
```

```

btn_elements = driver.find_elements(By.CSS_SELECTOR, "[id$='btn']")

print(f"\nElements whose id ends with 'btn': {len(btn_elements)}")

for b in btn_elements:

    print("->", b.get_attribute("id"), "| text:", b.text)

table_elements = driver.find_elements(By.CSS_SELECTOR, "[id*='table' i]")

print(f"\nElements whose id contains 'table': {len(table_elements)}")

for t in table_elements:

    print("->", t.get_attribute("id"))

day_checkboxes = driver.find_elements(By.CSS_SELECTOR, "input[type='checkbox'][id]")

print(f"\nCheckboxes that have a non-empty id attribute: {len(day_checkboxes)}")

for c in day_checkboxes:

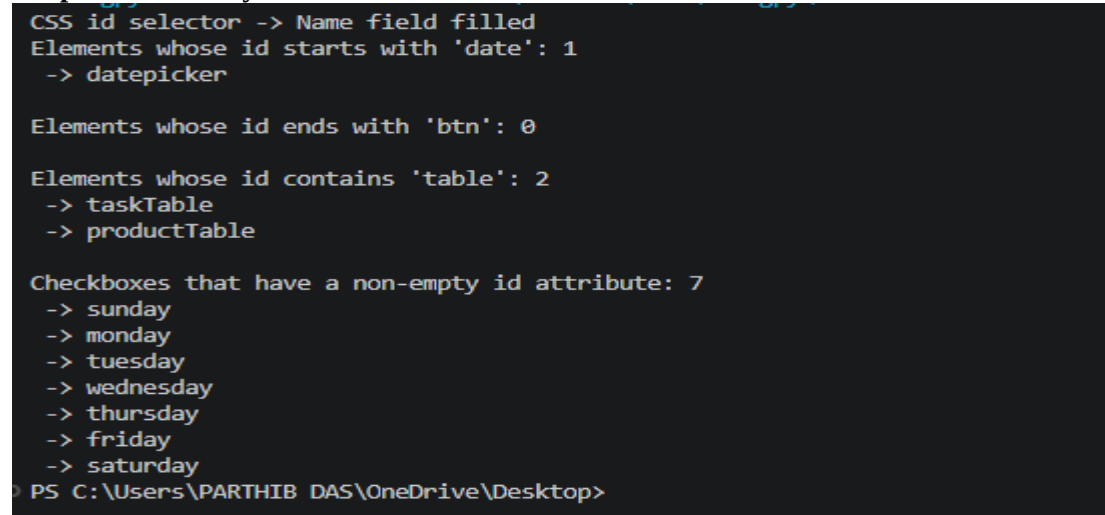
    print("->", c.get_attribute("id"))

time.sleep(2)

driver.quit()

```

Output with clearly labelled screenshots:



```

CSS id selector -> Name field filled
Elements whose id starts with 'date': 1
-> datepicker

Elements whose id ends with 'btn': 0

Elements whose id contains 'table': 2
-> taskTable
-> productTable

Checkboxes that have a non-empty id attribute: 7
-> sunday
-> monday
-> tuesday
-> wednesday
-> thursday
-> friday
-> saturday
PS C:\Users\PARTHIB DAS\OneDrive\Desktop>

```

Result: The child nodes were successfully identified using CSS selectors.

Observation: The > CSS selector is used to locate the direct child of a specified parent element.

Conclusion: The experiment demonstrated how CSS child selectors can be effectively used for precise web element identification in Selenium.